Multiprocessors

Grain size: bit, instruction, thread, inter-program

1. Execution model
   - SISD
   - SIMD
   - MIMD
   - SPMD
2. Physical location of memory
3. Types of interconnect
3. Memory access latency variability
4. Programmer's model of memory

Flynn's taxonomy

Single / Multiple Instruction streams

Single / Multiple Data streams

SISD, SIMD, MISD, MIMD

Single Program Multiple Data

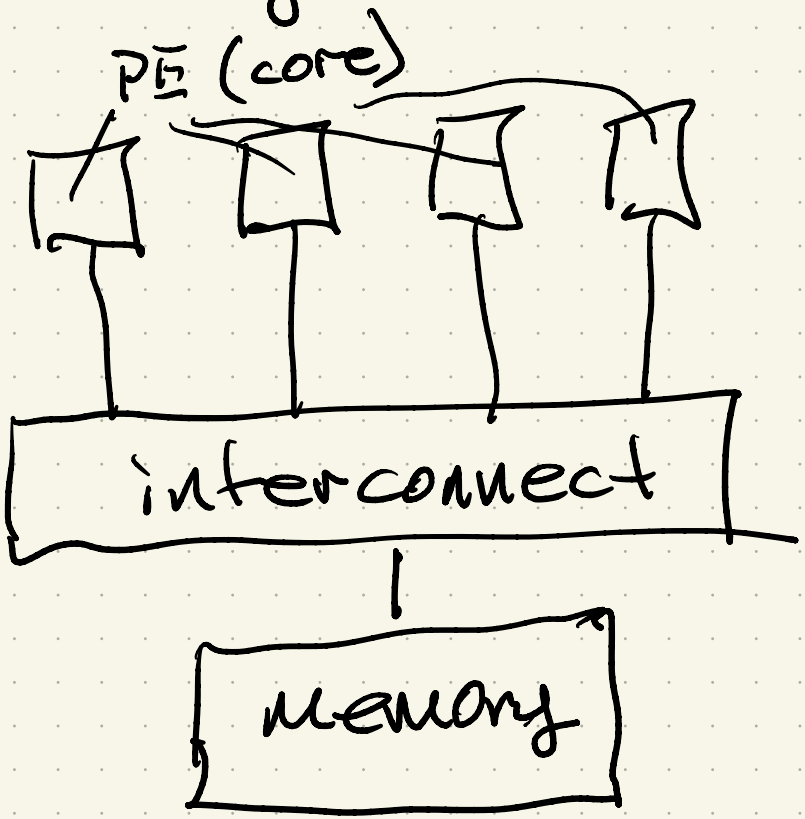
SPMD (SIMT)

1 thread, multiple copies

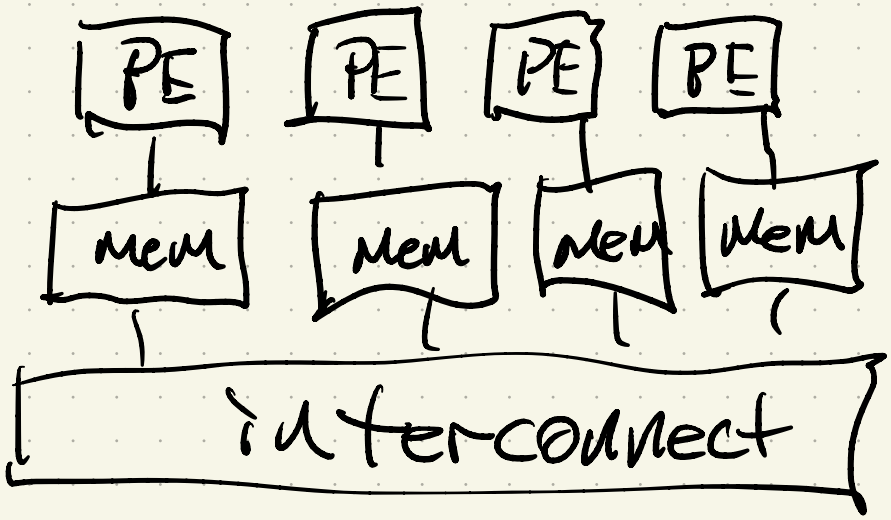
if.. then..else

No inter-thread synchronization

Physical memory location types



Centralized memory

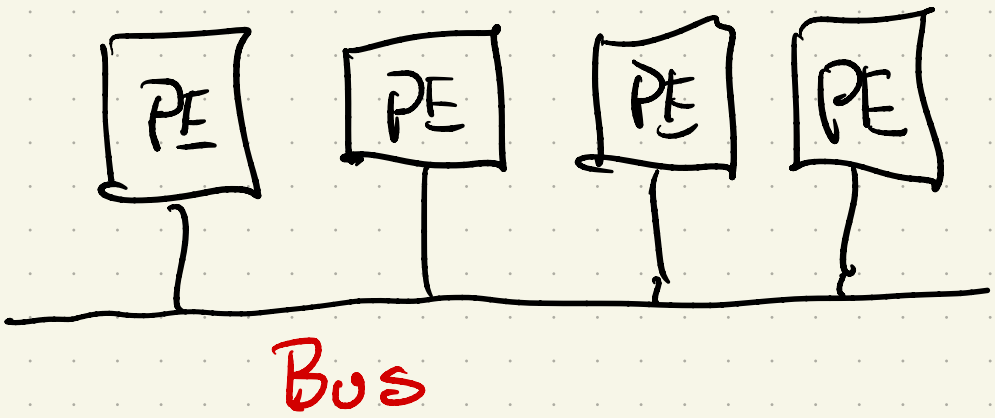


Distributed Memory

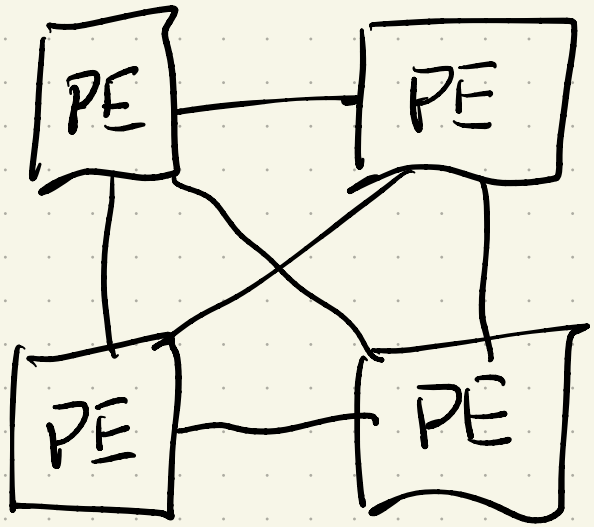
Types of interconnects

Shared/common

observable



not observable



Memory access latency Types

1. Symmetric Multiprocessors ("SMP")

   Uniform memory access latency ("UMA")

2. Non-uniform memory access latency

   NUMA

Programmer's model

OpenMP

Shared memory

- all threads have the same memory access space

MPI

Message passing

- Each thread has its own address space,
- programmer shares values via passing messages

256 #s, sum all numbers

4 PEs

Message passing

256

```
// Array is 1024 total size
// 4 processors
double A[64];
double mySum=0;
for(int i=0;i<64;i++)
  mySum+=A[i];
if(myPID=0){
  for(int p=1;p<4;p++){
    int pSum;
    recv(p,pSum);
    mySum+=pSum;
  }
  printf("Sum: %lf\n",mySum);
}else
  send(0,mySum);
```

Any # of PE's without hardware support

My PE id

0..3

Shared memory

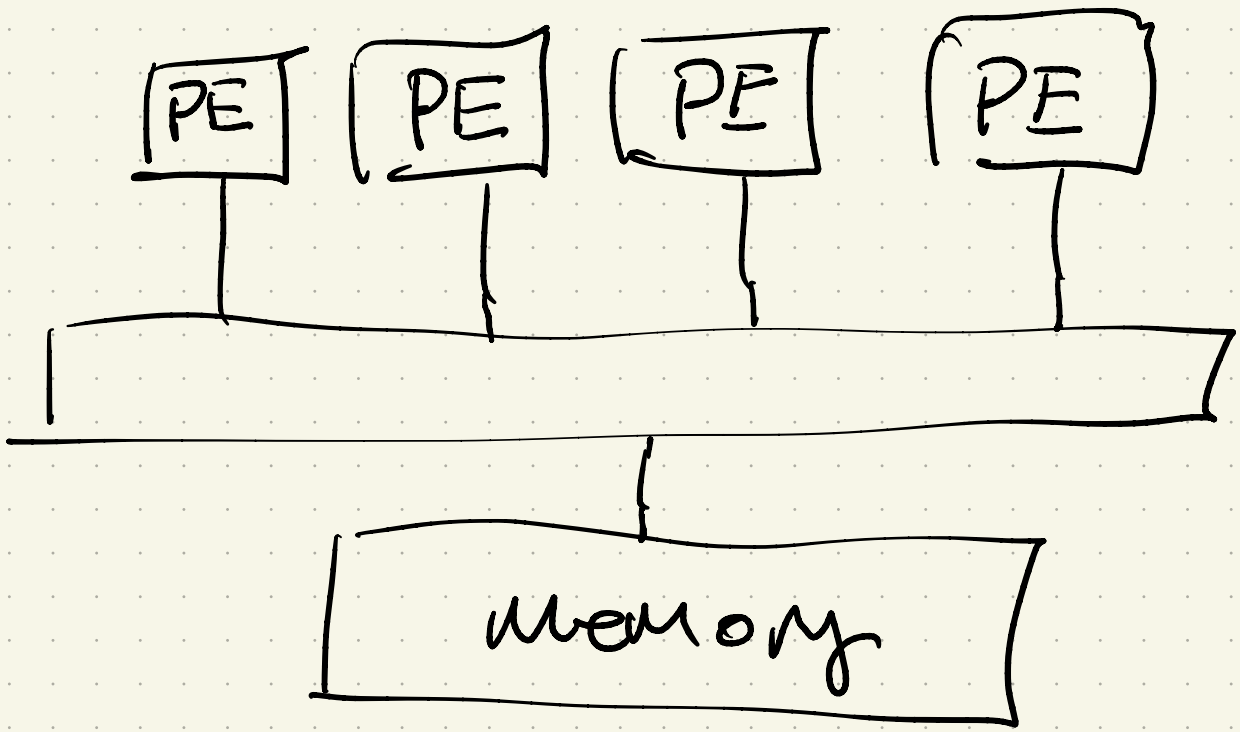
```
shared double A[1024];
shared double Sum=0;
shared mutex Lock;
double mySum=0;
for(int i=myPID*64;i<(myPID+1)*64;i++)
  mySum+=A[i];
lock(Lock);
Sum+=mySum;
unlock(Lock);
if(myPID=0)
  printf("Sum: %lf\n",Sum);
```

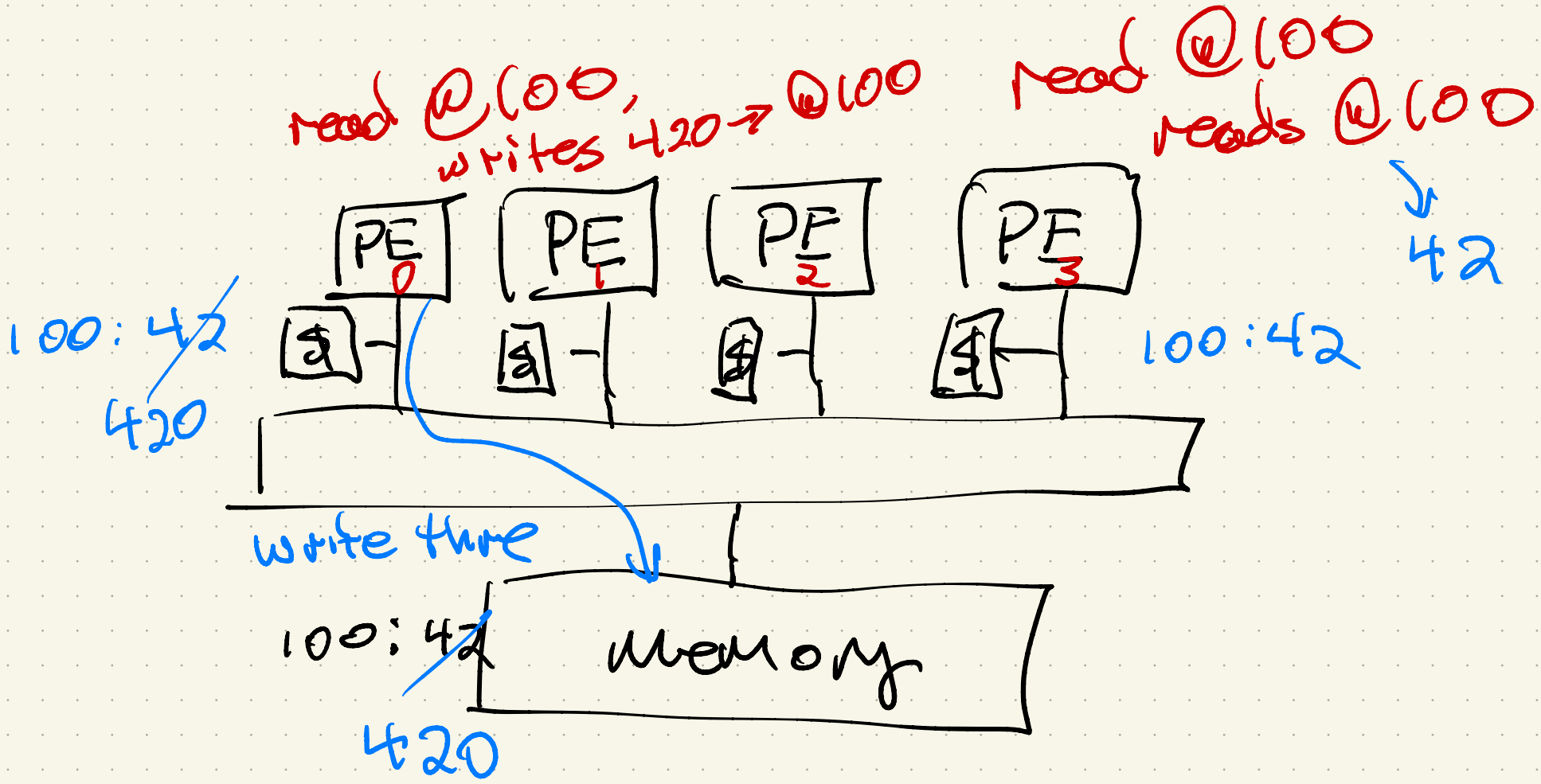
256

:) arch. independent Program

:(Hardware must implement shared memory

Centralized shared memory





Cache coherence problem

Coherent cache system

Means:

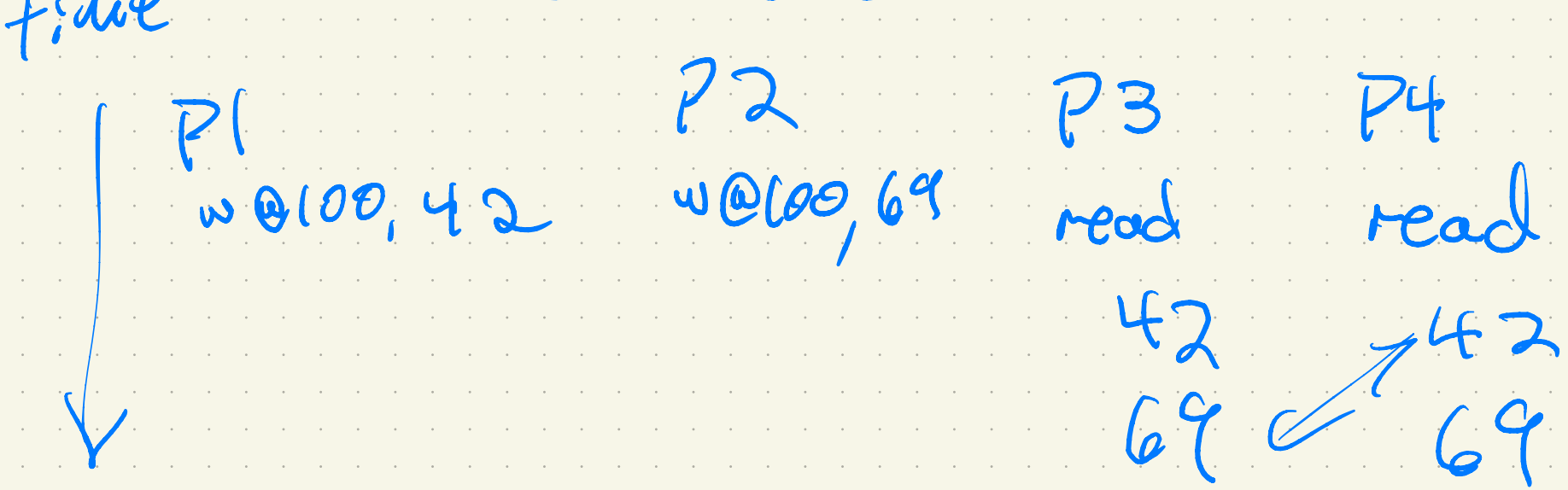
1. Read what is written

   A read of X on P1, return most recently written value of X by P1 if no other proc element written to X

2. Coherent writes seen by all processors (eventually)

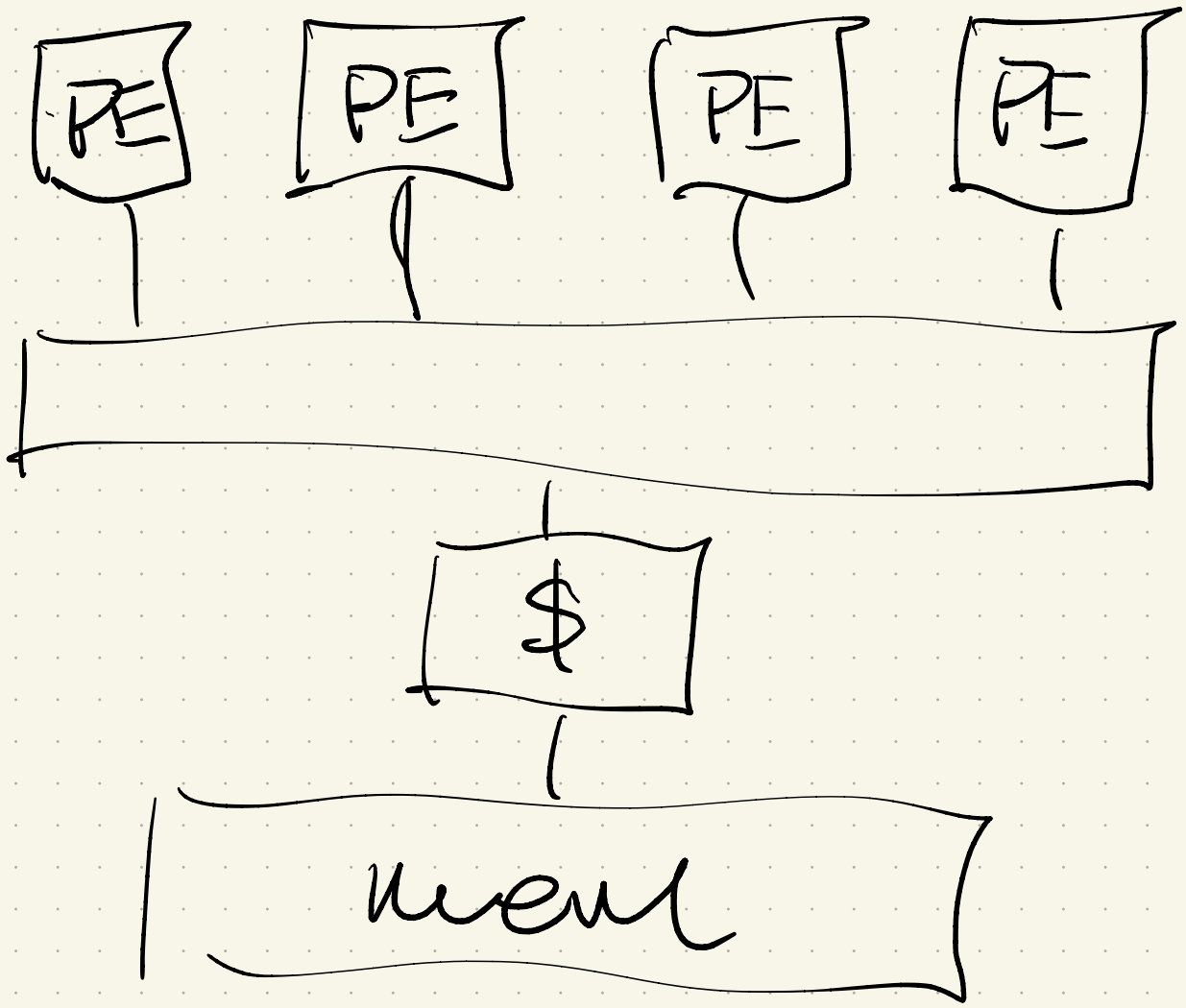
   if P1 writes to X and after sufficient time, P2 reads X, P2 gets value P1 wrote

3. Write causality: writes to the same location are serialized, and all PE's see the same order of writes

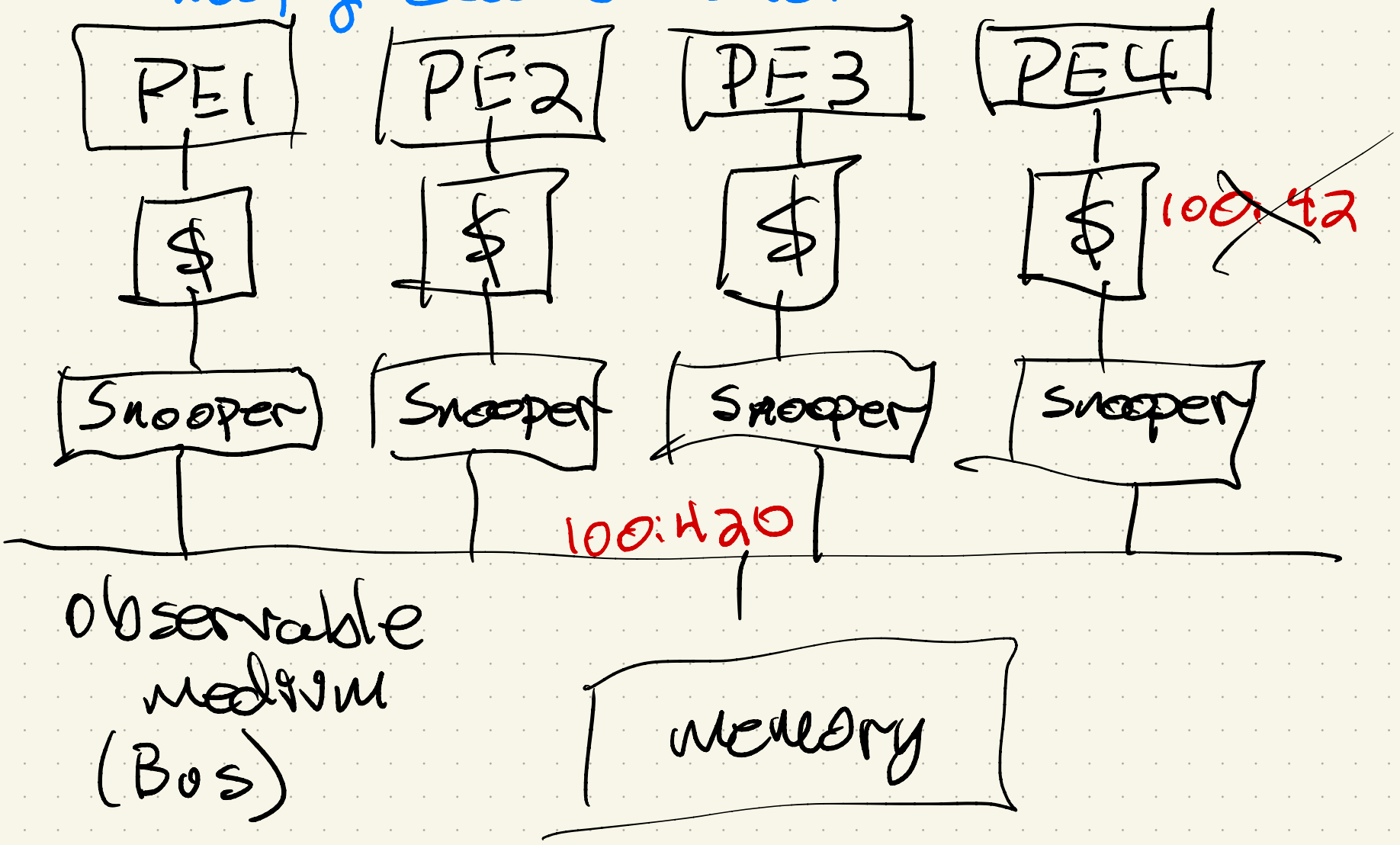


Cache coherence solutions

1. Shared cache — doesn't scale #PEs > 4



Snoopy cache coherence



Each $ has dirty bit, valid bit
+ shared bit

dirty	valid	shared	
X	0	X	"invalid" (I)
1	1	0	"Modified" (M)
0	1	1	"shared" (S)
0	1	0	"not shared"

MSI protocol